

GPU-Accelerated Simulated Annealing for VLSI Macro Placement

Sathvik Redrouthu

1 Background

Given a rectangular chip die, a set of rectangular macros, fixed pins, and a hypergraph netlist, the program will search for a legal macro placement that minimizes estimated wirelength and overlap.t evaluation on the GPU.

Macro placement is an important problem in chip physical design. In modern chips, large blocks such as SRAMs, analog blocks, accelerators, and IP modules must be placed on a two-dimensional die. The placement has to satisfy geometric constraints, such as keeping macros inside the die and avoiding overlap, while also optimizing objectives such as wirelength, routing congestion, and timing.

Input: we are given a rectangular die, a set of rectangular macros (design blocks), and a set of nets (wires) connecting those macros and fixed pins. For example:

```
{
  "die": {"width": 1000, "height": 1000},
  "macros": [
    {"name": "m0", "width": 100, "height": 80},
    {"name": "m1", "width": 120, "height": 90}
  ],
  "pins": [
    {"name": "p0", "x": 0, "y": 500}
  ],
  "nets": [
    ["m0", "m1", "p0"]
  ]
}
```

The goal is to assign each macro an (x, y) location such that the macros do not overlap and the estimated interconnect cost is low. This is well-suited for CUDA because many parts of the placement objective are parallelizable. For example, the half-perimeter wirelength of each net can be computed independently. Also, many simulated annealing chains can be run independently in parallel, allowing the GPU to explore many possible placements at once.

The program will first generate an initial random placement. The CPU baseline will improve the placement using simulated annealing. Then the CUDA implementation will run many annealing chains in parallel and return the best placement found.

2 Computation Details

Each placement consists of coordinates for N macros $P = \{(x_i, y_i)\}_{i=1}^N$ where macro i has width w_i and height h_i . The cost function will combine wirelength, overlap, and boundary penalties:

$$C(P) = \alpha \cdot \text{HPWL}(P) + \beta \cdot \text{OverlapPenalty}(P) + \gamma \cdot \text{BoundaryPenalty}(P)$$

The half-perimeter wirelength of a net n is:

$$\text{HPWL}(n) = \left(\max_{p \in n} x_p - \min_{p \in n} x_p \right) + \left(\max_{p \in n} y_p - \min_{p \in n} y_p \right)$$

The total wirelength is $\text{HPWL}(P) = \sum_{n \in \mathcal{N}} \text{HPWL}(n)$ where $\mathcal{N}$ is the set of nets.

The overlap penalty between two macros i and j can be computed using their rectangle intersection area. If macro i occupies $[x_i, x_i + w_i] \times [y_i, y_i + h_i]$ and macro j occupies $[x_j, x_j + w_j] \times [y_j, y_j + h_j]$, then their overlap area is:

$$\max(0, \min(x_i + w_i, x_j + w_j) - \max(x_i, x_j)) \cdot \max(0, \min(y_i + h_i, y_j + h_j) - \max(y_i, y_j))$$

The total overlap penalty is:

$$\text{OverlapPenalty}(P) = \sum_{i < j} \text{overlap}(i, j)$$

The simulated annealing algorithm proposes random moves such as moving one macro to a new location or swapping two macros. If the new placement has cost difference $\Delta C = C(P_{\text{new}}) - C(P_{\text{old}})$, then the move is accepted with probability: $p = \min(1, e^{-\Delta C/T})$ where T is the current temperature.

The CPU version will run a single simulated annealing chain. The CUDA version will run many independent chains in parallel. Each chain will start from a different random initial placement. At the end, the program will choose the best placement among all chains.

3 Results